

1 Questions à choix multiple

1.1 Questions à réponse unique

1. Quelle est la valeur de cette expression ?

`(12 = (4*3)) not`

A. true B. false C. error

2. Laquelle de ces phrases est vraie ?

A. Le point est obligatoire à la fin de chaque instruction
B. Le point est obligatoire pour séparer chaque instruction de la suivante

3. Quel type de paramètre devrait être passé en argument au message `ifFalse:` ?

A. n'importe quel booléen
B. Boolean
C. false
D. un bloc
E. n'importe quelle expression

4. Question : Dans Pharo, où commence le processus de recherche de méthode lorsqu'un message est envoyé à un objet ?

A. Dans la superclasse de l'objet
B. Dans la métaclasse de l'objet
C. Dans la classe de l'objet
D. Dans la classe racine (Object)

1.2 Questions à réponses multiples

1. Pourquoi l'envoi de message est le meilleur moyen d'implémenter un choix entre plusieurs alternatives ?

- ☐ parce que plusieurs méthodes dont le comportement dépend des alternatives peuvent être étendues en ajoutant une nouvelle classe
- ☐ parce que chaque méthode est plus longue
- ☐ parce qu'ajouter une nouvelle alternative n'implique pas de chercher dans le code des occurrences de `if/switch`
- ☐ parce que la sécurité est plus grande

2. Qu'est-ce que `nil` ?

- ☐ une variable spéciale
- ☐ une instance de `UndefinedObject`
- ☐ une instance de `False`
- ☐ une variable globale

3. Qu'est-ce que le double dispatch dans Pharo ?

- ☐ Une technique où une méthode est sélectionnée uniquement en fonction de la classe du récepteur.
- ☐ Un processus où la méthode à exécuter est déterminée par la classe à la fois du récepteur et de l'argument.
- ☐ Une manière de déléguer le comportement à une autre méthode en fonction de l'argument.

- ☐ Une méthode d'optimisation de l'envoi de messages entre objets.
4. Question : Lesquelles des affirmations suivantes à propos de la réflexivité dans Pharo sont vraies ?
- ☐ Les classes et méthodes dans Pharo peuvent être inspectées et modifiées à l'exécution.
 - ☐ Le méta-modèle dans Pharo permet aux objets d'être conscients de leur propre structure et comportement.
 - ☐ La réflexivité dans Pharo signifie que les objets ne peuvent pas modifier leurs propres définitions de classe.
 - ☐ Le concept de réflexivité est lié à la capacité d'accéder et de manipuler l'environnement d'exécution.

2 Objets et messages

2.1 Expressions

Pour chaque expression suivante, vous donnerez : (1) Quel est l'objet receveur ? (2) Quel est le selecteur du message ? (3) Quels sont les arguments ? Vous pouvez entourer proprement les expressions sur la feuille elle-même.

3 - 4

Time now

aDictionary at: idNumber put: 8159681596

Point selectors

[:x | x + 2] value: 2

2.2 Combien de messages envoyés dans les expressions

(ZnEasy getPng: 'https://tileopenstreetmap.org/8.png') asMorph openInWindow

announcer removeSubscription: self

(10@20 corner: 100@200) topCorner

3 Programmation

3.1 Exercice I

Donnez l'expression Pharo qui permet de :

1. filtrer les entiers positifs d'une collection `collection`,
2. récupérer les carrés des entiers positifs d'une collection `collection`,
3. rejeter les entiers positifs d'une collection `collection`.

3.2 Exercice II

Nous voulons travailler sur le prix des tickets de train. Dans les tickets de train il y a des places en seconde et en première. Les tarifs changent suivant la personne : enfant ou grand-mère. Voici quelques tests.

```
testPriceOfChildSecond
  self assert: (SecondClassTrainTicket new priceFor: Child new)
    equals: 35

testPriceOfGranmaSecond
  self assert: (SecondClassTrainTicket new priceFor: Granny new)
    equals: 45

testPriceOfChildFirst
  self assert: (FirstClassTrainTicket new priceFor: Child new)
    equals: 55

testPriceOfGranmaFirst
  self assert: (FirstClassTrainTicket new priceFor: Granny new)
    equals: 65
```

- Proposez une implémentation du calcul de ticket de train sans utilisation de conditions.
- Ajoutez un autre tarif pour les bus pour les enfants à 20 Euros en première et seconde classe.

4 Modèle objet

1. Que représente **self** et que représente **super** ? Quelle est la différence entre **self** et **super** ?
2. Donnez l'équivalent du diagramme de classe du code suivant, puis, donnez le nom complet de toutes les méthodes (classe + méthode) exécutées par les deux expressions suivantes : **C new foo** et **D new foo**, ainsi que leur résultat.

```
Object subclass: #A ... | A subclass: #B ...
B subclass: #C ...      | B subclass: #D ...
```

```
A>>#foo      | B>>#foo      | C>>#foo
  ^ self val  |   ^ super foo |   ^ super foo + 10
```

```
A>>#val      | B>>#val      | D>>#val
  ^ 42        |   ^ 10       |   ^ 50
```

5 AST et évaluation

Pour chaque expression suivante, vous donnerez : (1) le résultat de l'évaluation de l'expression ou une explications et (2) l'AST de l'expression.

Expression 1

```
| anArray |  
anArray := #('first' 'second' 'third' 'fourth').  
anArray at: 2
```

Expression 2

```
CompiledMethod allInstances select: [ :m | m selector beginsWith: 'test']
```

Expression 3

```
curry := [:x | [:y | x + y]].  
curry value: 4
```

6 Visiteur et réflexivité

6.1 Répondez aux questions

1. Définissez le pattern Visiteur. Donner un exemple.
2. Quelles sont les méthodes qui doivent être impérativement codées lorsque l'on code un pattern visiteur ?

6.2 Analysis — Détection des appels récursifs

Implémentez, en utilisant du pseudo-code ou du code Pharo, un visiteur qui parcourt l'AST d'une méthode Pharo et qui retourne `true` lorsque celle-ci effectue un appel récursif.

Dans l'exemple suivant, le visiteur retourne `true`.

```
foo: arg1  
...  
self foo: 45.
```

alors qu'appliqué sur la méthode suivante, le visiteur retourne `false`.

```
foo: arg1  
...  
arg1 foo: 33.  
...
```

Vous pouvez réutiliser `PharoVisitor` (`RBProgramNodeVisitor`) ou écrire le vôtre. Vous pouvez utiliser des classes de la hiérarchie de nœuds AST de Pharo à partir de la dernière page.

```
OCNode
  OCProgramNode
    OCommentNode
    OMethodNode
    OPragmaNode
    OReturnNode
    OSequenceNode
    OValueNode
      OAnnotationMarkNode
      OArrayNode
      OAssignmentNode
      OBlockNode
      OCascadeNode
      OLiteralNode
        OLiteralArrayNode
        OLiteralValueNode
      OMessageNode
      OSelectorNode
      OVariableNode
```

Figure 1: Aperçu de la hiérarchie des nœuds de méthode. L'indentation implique l'héritage.

Minimal Syntax (II)

Object constructors & reserved syntactic constructs		
"comment"		
'string'	sequence of characters	
#symbol	unique string	
\$a, Character space	two ways to create characters	
12 2r1100 16rC	twelve (decimal, binary, hexa)	
3.14 1.2e3	floating-point numbers	
#{abc 123}	literal array with the symbol #abc and the number 123	
{foo . 3+2}	dynamic array built from 2 expressions	
#[123 21 255]	byte array	
foo bar	declaration of two temporary variables	
var := expr	assignment	
exp1. exp2	period - statement separator	
;	semicolon - message cascade	
[:p expr]	code block with a parameter	
<unary>	method annotation	
<key: 'any' wrd: #lit>	with any literal arguments	
^ expr	caret - return/answer a result from a method	

Message Sending

When we send a message to an object (the *receiver*), the corresponding method is selected and executed, and the method answers an object. Message syntax mimics natural languages, with a subject, a verb, and complements.

Pharo	Java
aColor r: 0.2 g: 0.3 b: 0	aColor.setRGB(0.2,0.3,0)
d at: '1' put: 'Chocolate'.	d.put("1", "Chocolate");

Three Types of Messages: Unary, Binary, and Keyword

A **unary message** has no arguments.

Array new.	~~> anArray
#{4 2 1} size.	~~> 3

new is an unary message sent to classes (classes are objects).

A **binary message** takes only one argument and is named by one or more symbol characters from +, -, *, =, <, >, ...

3 + 4	~~> 7
'Hello', 'World'	~~> 'Hello World'

The + message is sent to the object 3 with 4 as argument. The string 'Hello' receives the message , (comma) with 'World' as the argument.

A **keyword message** can take one or more arguments that are inserted in the message name.

'Pharo' allButFirst: 2.	~~> 'aro'
[:x x + 2] value: 7	~~> 9
3 to: 10 by: 2.	~~> (3 to: 10 by: 2)

The second line executes a block. The third example sends to: by: to 3, with arguments 10 and 2; this returns an interval containing 3, 5, 7, and 9.

Message Precedence

Parentheses > unary > binary > keyword, and finally from left to right.

(15 between: 1 and: 2+4*3) not	~~> false
--------------------------------	-----------

Messages + and * are sent first, then between: and: is sent, and not. The rule suffers no exception: operators are just binary messages with *no notion of mathematical precedence*. 2+4*3 reads left-to-right and gives 18, not 14!

Cascade: Sending Multiple Messages to the Same Object

Using ; (a cascade) multiple messages are sent to the result of the same expression. Here ; arrives after add: 1, so messages add: 2 and add: 3 are sent to add: 1's receiver: a collection.

OrderedCollection new	
add: 1;	
add: 2;	
add: 3.	

The whole message cascade value is the value of the last message sent (here 3). To return the receiver of the message cascade instead (*i.e.*, the collection), send yourself as the last message of the cascade.

Blocks

Blocks are objects containing code that is executed on demand. They are the basis for control structures: conditionals & loops.

2 = 2	
ifTrue: [Error signal: 'Help'].	

Send the message ifTrue: to the boolean true (computed

from 2 = 2) with a block as argument. Because the boolean is true, the block is executed and an exception is signaled.

#{'Hello World' \$!)	
do: [:e Transcript show: e]	

Send the message do: to an array. This executes the block once for each element, passing it via the e parameter. As a result, Hello World! is printed.

Common Constructs

Conditionals	
condition	if (condition)
ifTrue: [action]	{ action(); }
ifFalse: [anotherAction]	else { anotherAction(); }

[condition] whileTrue:	while (condition) { action();
[action. anotherAction]	anotherAction(); }

Loops/Iterators	
1 to: 11 do: [:i	for(int i=1; i<11; i++){
Transcript show: i ; cr]	System.out.println(i); }
names	String [] names ={"A", "B", "C"};
names := #('A' 'B' 'C').	for(String name : names) {
names do: [:each	System.out.print(name);
Transcript show: each, ' , ']	System.out.print(","); }

Collections start at 1. aCol at: i accesses element at i and aCol at: i put: value sets element at i to value.

Collections	
#{4 2 1} at: 3	~~> 1
#{4 2 1} copy at: 3 put: 6	~~> #{4 2 6}
{4 . 2 . 1} at: 3 put: 6	~~> #{4 2 6}
(Array new: 2) add: 4; add: 2 ; yourself	~~> #{4 2}
Set new add: 4; add: 4 ; yourself	~~> aSet(4)
Dictionary new	
at: #a put: 'Alpha' ; yourself	~~> a Dictionary(#a->'Alpha')